Tasks for session 2

Task 1: Constants in Python

Introduction

A constant is a value that should not be changed during the execution of a program. Many programming languages provide a special keyword such as `const` to create constants. However, Python does not have a built-in `const` keyword. Instead, Python developers use different methods and conventions to represent constant values.

Method 1: Using UPPER_CASE Naming Convention

How It Works

The most common way to define constants in Python is by using uppercase variable names. This is a naming convention recommended by PEP 8, Python's official style guide.

Python does not prevent changing these variables, but uppercase names indicate that the value should be treated as a constant.

Code Example

```
PI = 3.1415926535
MAX_USERS = 100

print(PI)
```

The value can still be modified:

```
PI = 4
```

Advantages

- Simple and easy to understand.
- Does not require extra libraries.
- Follows Python coding standards.

- Commonly used by developers.

Disadvantages

- It is not a real constant.
- The value can still be changed accidentally.
- Depends on programmer discipline.

When to Use

Use this method for:

- Mathematical constants.
- Configuration values.
- Small and medium projects.

Method 2: Using Frozen Dataclass

How It Works

Python provides the `dataclasses` module that allows developers to create immutable objects using `frozen=True`.

A frozen dataclass prevents modification after the object is created.

Code Example

```
from dataclasses import dataclass

@dataclass(frozen=True)
class Constants:
    PI = 3.14159
    GRAVITY = 9.81

print(Constants.PI)
```

Advantages

- Provides immutability.
- Groups related constants together.
- Useful in large applications.

Disadvantages

- Requires more code.
- Not necessary for simple constants.
- Only works with dataclass objects.

When to Use

Use it for:

- Large projects.
- Object-oriented applications.
- Systems requiring stronger protection.

Method 3: Using Enum

How It Works

The `Enum` module creates a collection of constant values with meaningful names.

Code Example

```
from enum import Enum

class Status(Enum):
    SUCCESS = 1
    FAILED = 2
    PENDING = 3

print(Status.SUCCESS)
```

Advantages

- Prevents accidental changes.
- Improves readability.
- Good for fixed choices.

Disadvantages

- Not suitable for mathematical constants.
- Requires additional syntax.

When to Use

Use Enum for:

- Status values.
- Error codes.
- User roles.
- Predefined options.

Method 4: Using a Constants Module

How It Works

Developers can create a separate Python file that contains all constant values and import it into other files.

Example:

constants.py

```
PI = 3.14159
GRAVITY = 9.81
MAX_SPEED = 120
```

main.py

```
import constants

print(constants.PI)
```

Advantages

- Keeps projects organized.
- Easy to maintain.
- Allows sharing constants between files.
- Suitable for large applications.

Disadvantages

- Values can still be modified.
- Does not provide real protection.

When to Use

Use it for:

- Large software projects.
- Shared configuration values.
- Multi-file applications.

Method 5: Using a Custom Constant Class

How It Works

A custom class can be created to prevent changing values by controlling attribute assignment.

Code Example

```
class Constants:
    PI = 3.14159

    def __setattr__(self, name, value):
        raise AttributeError("Constants cannot be modified")

constant = Constants()

print(constant.PI)
```

Advantages

- Strong protection against modification.
- Can be customized.
- Useful for special cases.

Disadvantages

- More complicated.
- Requires extra programming.
- Rarely used in normal projects.

When to Use

Use it for:

- Critical applications.
- Systems requiring strict control.

Comparison of Python Constant Methods

Method	True Constant	Ease of Use	Recommended
UPPER_CASE Naming	No	Very Easy	Yes
Frozen Dataclass	Yes	Easy	Yes
Enum	Yes	Easy	Yes
Constants Module	No	Very Easy	Yes
Custom Class	Yes	Difficult	Sometimes

Recommendation

For most real-world Python projects, the best approach is using **UPPER_CASE naming with a dedicated constants module**.

This approach is simple, readable, follows Python standards, and is widely used in professional software development.

For applications that require real immutability, developers should use:

- `Enum`
- `@dataclass(frozen=True)`

because they provide better protection against accidental modification.

Task 2: Programming Languages Without const

Introduction

Some programming languages do not provide a native `const` keyword or do not enforce true constants. Developers use different techniques to create values that should not change.

---

1. Python

Approach

Python uses naming conventions and special structures instead of a `const` keyword.

Example

```
PI = 3.14159
```

Developers understand that uppercase variables should not be modified.

Advantages

- Simple.
- Easy to read.
- Python standard practice.

Disadvantages

- Not enforced by the language.

---

2. Ruby

Approach

Ruby uses uppercase variable names as constants.

Example

```
PI = 3.14159
```

Ruby treats uppercase variables as constants, but modification only generates a warning.

Advantages

- Easy syntax.

- Built into the language.

Disadvantages

- Does not completely prevent modification.

---

3. Lua

Approach

Lua does not have constants. Developers usually use local variables and avoid changing them.

Example

```
local PI = 3.14159
```

Advantages

- Simple implementation.
- Lightweight.

Disadvantages

- No true protection.

---

4. JavaScript (Before ES6)

Approach

Older JavaScript versions did not have the `const` keyword, so developers used naming conventions.

Example

```
var PI = 3.14159;
```

Modern JavaScript introduced:

```
const PI = 3.14159;
```

Advantages

- Simple approach.

Disadvantages

- Old versions allowed modification.

---

5. MATLAB

Approach

MATLAB creates constants using classes with constant properties.

Example

```
classdef Constants
    properties (Constant)
        PI = 3.14159
    end
end
```

Advantages

- Provides real constant behavior.
- Organized structure.

Disadvantages

- Requires more code.

Comparison with Languages Supporting const

Language	Native Constant Support	Method Used
Python	No	Naming convention, Enum
Ruby	Partial	Uppercase variables
Lua	No	Local variables
JavaScript (old versions)	No	Naming convention
MATLAB	No	Constant classes
C++	Yes	const keyword
Java	Yes	final keyword
C#	Yes	const keyword

Task 3: Floating-Point Numbers

Introduction

Floating-point numbers are used by computers to represent real numbers. They allow programs to handle very large and very small values efficiently. However, floating-point numbers can suffer from precision problems because computers store numbers in binary format.

How Floating-Point Numbers Are Stored (IEEE 754)

Most computers follow the IEEE 754 standard for floating-point representation.

A floating-point number consists of three parts:

```
Sign Bit | Exponent | Fraction (Mantissa)
```

For a 64-bit double precision number:

Component	Number of Bits
Sign	1 bit
Exponent	11 bits
Fraction	52 bits

The value is calculated using:

```
(-1)^Sign × 1.Fraction × 2^(Exponent-Bias)
```

Why Decimal Numbers Cannot Always Be Represented Exactly

Computers use binary numbers (base 2), while humans use decimal numbers (base 10).

Some decimal values cannot be represented exactly in binary.

Example:

```
0.1 (decimal)
```

becomes a repeating binary fraction:

```
0.000110011001100...
```

Since the computer has limited storage, it stores an approximation.

Example of Floating-Point Error

Python example:

```
print(0.1 + 0.2)
```

Output:

```
0.30000000000000004
```

The result is slightly different because the numbers are stored as approximations.

Common Floating-Point Problems

1. Rounding Errors

Example:

```
round(2.675, 2)
```

Output:

```
2.67
```

---

2. Comparison Problems

Example:

```
0.1 + 0.2 == 0.3
```

Output:

```
False
```

---

3. Accumulated Errors

Small errors can increase after many calculations.

Example:

```
total = 0

for i in range(100):
    total += 0.1

print(total)
```

The result may not be exactly 10.

Techniques to Avoid Floating-Point Problems

1. Using Decimal

The Decimal module provides accurate decimal arithmetic.

Example:

```
from decimal import Decimal

x = Decimal("0.1")
y = Decimal("0.2")
```

```
print(x + y)
```

Output:

```
0.3
```

Used for:

- Banking.
- Financial calculations.
- Accounting systems.

---

2. Using Fraction

Fractions provide exact rational number calculations.

Example:

```
from fractions import Fraction

x = Fraction(1,10)
y = Fraction(2,10)

print(x+y)
```

Used for:

- Mathematical applications.
- Exact calculations.

---

3. Epsilon Comparison

Instead of comparing floating numbers directly:

```
a == b
```

Use:

```
abs(a-b) < 1e-9
```

or:

```
import math
```

```
math.isclose(a, b)
```

Used for:

- Scientific computing.
- Engineering simulations.

Real-World Examples

Banking Systems

Money calculations require exact values. Floating-point numbers can create incorrect balances, so systems use Decimal.

Scientific Applications

Scientific simulations may perform millions of calculations, so small errors must be controlled.

GPS and Navigation

Location calculations require careful handling of numerical precision.

Best Practices

- Avoid using `==` for floating-point comparisons.
- Use `Decimal` for financial applications.
- Use `Fraction` when exact fractions are needed.
- Use `math.isclose()` for comparing approximate values.
- Understand that floating-point numbers are approximations.